

Devoir surveillé d'informatique

⚠ Consignes

- Les programmes demandés doivent être écrits en C ou en OCaml. Dans le cas du C, on suppose que les bibliothèques standards usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`, `<stdassert.h>`, ...) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veuillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Questions de cours

Q1– Donner la définition d'une relation d'ordre sur un ensemble E .

Une relation $\mathcal{R}$ sur un ensemble E est une relation binaire sur E ayant les 3 propriétés suivantes :

1. *réflexive* : pour tout $x \in E$, $x \mathcal{R} x$.
2. *antisymétrique* : pour tout $(x, y) \in E^2$, si $x \mathcal{R} y$ et $y \mathcal{R} x$ alors $x = y$
3. *transitive* : pour tout $(x, y, z) \in E^3$, si $x \mathcal{R} y$ et $y \mathcal{R} z$ alors $x \mathcal{R} z$.

Q2– Montrer que la relation $\preccurlyeq$ sur $\mathbb{N}^2$ définie par $(x, y) \preccurlyeq (x', y')$ si et seulement si $x < x'$ ou $x = x'$ et $y \leq y'$ est une relation d'ordre sur $\mathbb{N}^2$. Comment appelle-t-on cette relation d'ordre ?

- Soit $(x, y) \in \mathbb{N}^2$, alors $(x, y) \preccurlyeq (x, y)$ car $x = x$ et $y \leq y$, donc $\preccurlyeq$ est bien réflexive.
- Soient $(x, y) \in \mathbb{N}^2$ et $(x', y') \in \mathbb{N}^2$ tels que $(x, y) \mathcal{R} (x', y')$ et $(x', y') \mathcal{R} (x, y)$, alors on a nécessairement $x = x'$ (dans le cas contraire on aurait $x < x'$ et $x' < x$ ou $x' = x$ ce qui est impossible). Donc, $y \leq y'$ et $y' \leq y$ c'est à dire $y = y'$.
- Soient $(x, y) \in \mathbb{N}^2$, $(x', y') \in \mathbb{N}^2$ et $(x'', y'') \in \mathbb{N}^2$ tels que $(x, y) \mathcal{R} (x', y')$ et $(x', y') \mathcal{R} (x'', y'')$, montrons par disjonction de cas que $(x, y) \mathcal{R} (x'', y'')$
 - $x < x'$ et $x' < x''$ alors $x < x''$ et donc $(x, y) \mathcal{R} (x'', y'')$
 - $x < x'$ et $x' = x''$ alors $x < x''$ et donc $(x, y) \mathcal{R} (x'', y'')$
 - $x = x'$ et $x' < x''$ alors $x < x''$ et donc $(x, y) \mathcal{R} (x'', y'')$
 - $x = x'$ et $x' = x''$ alors $y \leq y'$ et $y' \leq y''$, et donc $y \leq y''$ donc $(x, y) \mathcal{R} (x'', y'')$

La relation $\mathcal{R}$ est donc bien une relation d'ordre sur $\mathbb{N}^2$.

Q3– On dit qu'une relation d'ordre sur un ensemble E est bien fondée lorsqu'il n'existe pas de suites strictement décroissantes d'éléments de E . Montrer qu'une relation d'ordre sur un ensemble E est bien fondée si et seulement si toute partie non vide de E admet un élément minimal.

- Pour montrer le sens direct, on utilise la contraposée. Supposons qu'il existe une partie non vide F de E sans élément minimal. Puisque F est non vide on peut choisir $f_0 \in F$ et construire la suite $(f_n)_{n \in \mathbb{N}}$ par récurrence en définissant f_n ($n \geq 1$) comme un élément de F strictement inférieur à f_{n-1} cela possible puisque f_n n'est pas un élément minimal de F . Par construction (f_n) est alors une suite strictement décroissante d'éléments de F .
- Pour le sens réciproque, on utilise encore la contraposée. Supposons qu'il existe une suite $(f_n)_{n \in \mathbb{N}}$ strictement décroissante d'éléments de F . On considère l'ensemble $A = \{f_n, n \in \mathbb{N}\}$ alors A est non vide (par exemple $f_0 \in A$) et A ne possède pas d'élément minimal (si tel était le cas cet élément minimal serait un f_k avec $k \in \mathbb{N}$ or $f_{k+1} < f_k$ et $f_{k+1} \in A$ ce qui contredit la minimalité de f_k dans A .)

□ **Exercice 2** : *Terminaison de la fonction d'Ackerman*

On considère la fonction d'Ackerman $a : \mathbb{N} \times \mathbb{N} \mapsto \mathbb{N}$ définie par :

$$\begin{cases} a(0, m) &= m + 1 \\ a(n, 0) &= a(n - 1, 1) \text{ si } n > 0 \\ a(n, m) &= a(n - 1, a(n, m - 1)) \text{ si } n > 0 \text{ et } m > 0 \end{cases}$$

Q4– Calculer $a(1, 2)$

$$\begin{aligned} a(1, 2) &= a(0, a(1, 1)) \\ &= a(0, a(0, a(1, 0))) \\ &= a(0, a(0, a(0, 1))) \\ &= a(0, a(0, 2)) \\ &= a(0, 3) \\ &= 4 \end{aligned}$$

Q5– Prouver la terminaison de la fonction **ack** on précisera soigneusement le variant et la relation d'ordre bien fondée utilisée.

On considère l'ordre lexicographique sur N^2 noté $\preceq_L$, montrons que (n, m) est un variant .

- si $m = 0$ alors on effectue un appel récursif avec $(n - 1, 1)$ et comme $(n - 1, 1) \preceq_L (n, 0)$ la quantité (n, m) décroît strictement.
- sinon, on effectue un premier appel récursif avec $(n, m - 1)$ et comme $(n, m - 1) \preceq_L (n, m)$ la quantité (n, m) décroît strictement. Le second appel récursif s'effectue avec $(n - 1, a(n, m - 1))$ qui est strictement inférieur à (n, m) .

Dans tous les cas (n, m) décroît strictement à chaque appel récursif, c'est donc un variant et puisque $(N^2, \preceq)$ est un ensemble bien fondé, la fonction **ack** termine.

Q6– Ecrire en OCaml une fonction récursive **ack**: `int -> int -> int` qui prend en argument deux entiers positifs n et m et renvoie $a(n, m)$.

```
1 let rec ack n m =
2   match n, m with
3   | 0, m -> m + 1
4   | n, 0 -> ack (n - 1) 1
5   | n, m -> ack (n - 1) (ack n (m - 1));;
```

□ **Exercice 3** : *Recherche des k premiers maximums d'une liste*

Les fonctions demandées dans cet exercice doivent être écrites en langage OCaml.

On s'intéresse au problème de la recherche des k premiers maximums d'une liste de n entiers. Dans toute la suite de l'exercice on supposera que la liste est *non vide* : $n > 0$ et qu'on extrait moins de maximums qu'il n'y a d'éléments dans la liste c'est à dire que $k \leq n$. On cherche donc à écrire une fonction **kmax** : `int list -> int -> int list` qui renvoie la liste *triée dans l'ordre décroissant* des k premiers maximums de la liste donnée en argument . Par exemples :

- **kmax** [2; 5; 1; 8; 3; 0; 4] 2 renvoie [8, 5]
- **kmax** [7; 8; 8; 1; 6; 3; 2; 9] 3 renvoie [9; 8; 8]
- **kmax** [1; 0; 1; 2; 4] 4 renvoie [4; 2; 1; 1]

Les trois parties sont indépendantes et dans chacune d'elle on propose un algorithme différent afin d'écrire la fonction **kmax**.

■ **Partie I** : Résolution par recherche successive des maximums

Q7– Ecrire une fonction **max_reste** : `int list -> int * int list` qui prend en argument une liste et renvoie le couple composé du maximum de cette liste et de cette liste privée d'un de ses maximums. Par exemple **max_reste** [2; 6; 4; 6; 5] renvoie (6, [2; 4; 6; 5]). On pourra procéder par correspondance de motif et traiter le cas de la liste vide par un **failwith**.

```

1 let rec max_reste lst =
2   match lst with
3   | [] -> failwith "Extraction du maximum d'une liste vide"
4   | h::[] -> h, []
5   | h::t -> let m,r = max_reste t in if h>m then h, m::r else m, h::r;;

```

Q8– Donner en la justifiant brièvement la complexité de la fonction `max_reste`.

Le fonction `max_reste` a une complexité linéaire en fonction de la taille n de la liste car elle effectue $n - 1$ appels récursifs et chacun de ces appels n'effectue que des opérations élémentaires.

Q9– Ecrire une première version de la fonction `kmax` qui procède par extraction successive des k premiers maximums en utilisant la fonction `max_reste`. On procèdera de façon récursive sans utiliser les aspects impératifs de OCaml.

```

1 let rec kmax1 lst k =
2   (*Renvoie les k premiers maximums de la liste lst ainsi que la liste lst privée
   → de ces k premiers maximums*)
3   if k=0 then [],lst else(
4     let max, reste = max_reste lst in
5     let rmax, rlst = kmax1 reste (k-1) in
6     max::rmax, rlst);;

```

Q10– Quelle est la complexité de cette version de la fonction `kmax` en fonction du nombre k de maximums à extraire et de la longueur n de la liste ?

La fonction `kmax` effectue un appel récursif à `max_reste` pour chacun des k maximums à extraire et comme `max_reste` a une complexité en $\mathcal{O}(n)$, cette version de `kmax` a une complexité en $\mathcal{O}(kn)$.

■ Partie II : Résolution par un tri

Q11– Ecrire une fonction `kpremiers int list -> int -> int list` qui prend en argument une liste `lst` et un entier `k` et renvoie la liste composée des k premiers éléments de `lst`. Dans le cas où k est plus grand que la taille de la liste, on renvoie la liste entière. Par exemple `kpremiers [2; 7; 1; 8; 5] 3` renvoie la liste `[2; 7; 1]` et `kpremiers [12; 1; 9] 7` renvoie `[12; 1; 9]`. On procèdera de façon récursive sans utiliser les aspects impératifs de OCaml.

```

1 let rec kpremiers lst k =
2   match lst,k with
3   | _,0 -> []
4   | [],_ -> failwith "Pas assez d'éléments"
5   | h::t, k -> h::(kpremiers t (k-1));;

```

Q12– On propose d'écrire la fonction `kmax` en triant la liste par ordre décroissant puis en prenant ses k premiers éléments. En supposant que l'algorithme de tri utilisé a une complexité en $\mathcal{O}(n \log n)$, donner la complexité de ce nouvel algorithme en fonction du nombre k de maximums à extraire et de la longueur n de la liste.

La fonction `kmax` effectue un appel à la fonction de tri qui a une complexité en $\mathcal{O}(n \log n)$ et ensuite elle effectue un appel à la fonction `kpremiers` qui a une complexité en $\mathcal{O}(k)$. Donc la complexité de cette version de `kmax` est en $\mathcal{O}(n \log n + k)$ et comme $k < n$ la complexité est en $\mathcal{O}(n \log n)$.

- Q13**– La documentation du module `List` de Ocaml indique que la fonction `sort : ('a -> 'a -> int) -> 'a list -> 'a list` : « *Sort a list in increasing order according to a comparison function. The comparison function must return 0 if its arguments compare as equal, a positive integer if the first is greater, and a negative integer if the first is smaller.* ». Ecrire une implémentation de l'algorithme décrit à la question précédente et qui trie la liste à l'aide de `List.sort`.

```

1 let rec kmax2 lst k=
2   let lst_trie = List.sort (fun n m -> m-n) lst in
3   kpremiers lst_trie k;;

```

■ Partie III : Résolution en utilisant un tas

Dans la suite on suppose que la structure de données de tas d'entiers (type `int`), est *déjà implémentée* par un type `tas` sur lequel on dispose des fonctions suivantes :

- `cree_tas : int -> tas` qui prend en argument un entier `cap` et renvoie un tas binaire vide de capacité maximale `cap`.
 - `donne_taille : tas -> int` qui prend en argument un tas et renvoie sa taille (le nombre d'éléments actuellement stocké dans le tas).
 - `insere : int -> tas -> unit` qui insère une nouvelle valeur dans le tas. Cette fonction échoue lorsque le tas est plein.
 - `donne_min : tas -> int` qui renvoie la valeur minimale contenu dans le tas *sans modifier le tas*. Cette fonction échoue lorsque le tas est vide.
 - `extraite_min : tas -> int` qui renvoie, *en le supprimant du tas* le minimum du tas. Cette fonction échoue lorsque le tas est vide.
- Q14**– Rappeler en les justifiant, les complexités des opérations `insere` et `extraite_min` en fonction de la taille du tas notée k si on suppose que l'implémentation de la structure de tas est réalisée grâce à un tableau.

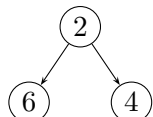
Un tas est un arbre binaire complet et à chaque étape d'une insertion ou d'une extraction, on remonte (ou on descend) d'un niveau dans cet arbre, la complexité de ces opérations est donc en $O(h)$ où h est la hauteur de l'arbre or l'arbre étant complet $O(h) = O(\log k)$ où k est la taille de l'arbre. En conclusion, `insere` et `extraite_min` ont une complexité en $O(\log k)$

Afin d'extraire les k premiers éléments d'une liste de taille n , on propose créer un tas de taille k puis de parcourir récursivement la liste, pour chaque élément rencontré :

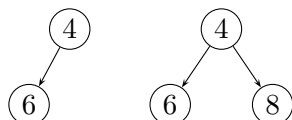
- si le tas n'est pas plein on y insère l'élément.
- sinon, on compare l'élément avec le minimum du tas, s'il est plus grand on extrait le minimum du tas et on insère l'élément dans le tas.

Lorsque tous les éléments de la liste ont été traités, le tas contient les k premiers maximums et il suffit de les extraire successivement du tas.

Par exemple, si on veut extraire les 3 premiers maximums de la liste `[4; 6; 2; 8; 3; 7; 1; 9; 5]`, après l'insertion des trois premiers éléments, le tas est :

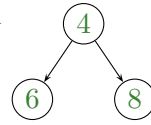


A l'étape suivante, 8 étant plus grand que 2 (le minimum du tas), on extrait 2 du tas et on y insère 8 ce qui donne :

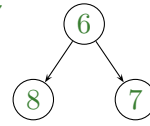


Q15– Poursuivre le déroulement de cet algorithme en faisant figurer comme ci-dessus les étapes de l'évolution du tas.

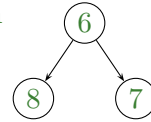
— On traite 3, comme $3 < 4$ pas de modification



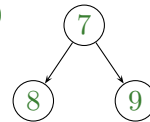
— On traite 7, comme $7 > 4$, on extrait 4 et on insère 7



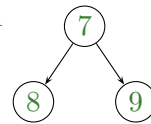
— On traite 1, comme $1 < 6$, pas de modification



— On traite 9, comme $9 > 6$, on extrait 6 et on insère 9



— On traite 5, comme $5 < 7$, pas de modification



Q16– Donner une implémentation de la fonction `kmax` utilisant ce nouvel algorithme. On rappelle qu'on pourra utiliser les fonctions de manipulation de la structure de `tas` données en début de partie. Comme précédemment, on procédera par récurrence sans utiliser les aspects impératifs de OCaml.

```

1  let kmax3 lst k=
2    let th = cree_tas k in
3    let rec aux lst =
4      match lst with
5        (* Si la liste est vide, tous les éléments ont été traités, les k maximums
           → sont les k éléments du tas*)
6        | [] -> List.init k (fun n-> extract_min th)
7        (* Sinon si le tas n'est pas plein on insère l'élément rencontré*)
8        | h::t -> if get_size th < k then (insert h th; aux t) else
9          (* Sinon on compare l'élément rencontré avec le minimum du tas, si l'élément
             → est plus grand on remplace le minimum par l'élément rencontré*)
10         (* Sinon on ne fait rien et on continue à traiter la liste*)
11         (let mt = get_min th in
12          if h>mt then (ignore (extract_min th); insert h th; aux t) else (aux t))
13    in
14    aux lst;;
  
```

Q17– Donner en la justifiant la complexité de ce nouvel algorithme en fonction de k et n .

Les opérations d'insertion et d'extraction dans le tas sont toutes en $O(\log k)$ car le tas est de taille k . On effectue ces opérations au plus n fois (une fois pour chaque élément du tableau) et donc la complexité de ce nouvel algorithme est $O(n \log k)$

□ Exercice 4 : Saut de valeur maximale

🎓 CAPES NSI 2023

Les fonctions demandées dans cet exercice sont à écrire en langage C.

Dans un tableau de flottants (type `double` du langage C) `tab` de taille `n`, on appelle *saut* un couple (i, j) avec $0 \leq i \leq j < n$ et la *valeur* d'un saut est la valeur `tab[j]-tab[i]`. Le but de l'exercice est de rechercher la

valeur maximale d'un saut dans un tableau. Par exemple, dans le tableau $\{ 2.0, 0.2, 3.0, 5.3, 2.0 \}$, la valeur maximale d'un saut est $5.3 - 0.2 = 5.1$, cette valeur est obtenue en considérant le saut $(1,3)$.

■ **Partie I** : Questions préliminaires et résolution naïve

- Q18**– Ecrire une fonction de signature `double valeur(double tab[], int i, int j, int n)` qui prend en argument un tableau (de `double`) `tab` de taille `n` ainsi que deux indices `i` et `j` et renvoie la valeur du saut (i,j) . On vérifiera les préconditions sur `i` et `j` à l'aide d'instructions `assert`. Par exemple si le tableau `tab` est $\{2.0; 0.2; 3.0; 5.3; 2.0\}$ alors `valeur(tab, 0, 2, 5)` renvoie `1.0` (car `tab[2]-tab[0] = 1.0`).

```

1 double valeur(double tab[], int i, int j, int n)
2 {
3     assert(0 <= i && i <= j && j < n);
4     return tab[j] - tab[i];
5 }
```

- Q19**– Donner un exemple de tableau avec exactement deux sauts de valeur maximale et préciser ces sauts.

La liste $[2, 6, 1, 5]$ possède deux sauts de valeurs maximale : $(0,1)$ et $(2,3)$ (ces deux sauts ont une valeur de 4)

- Q20**– À l'aide d'un contre-exemple, montrer qu'on ne peut pas se contenter de chercher le minimum et le maximum d'un tableau pour trouver un saut de valeur maximale.

Dans la liste $[2, 6, 1, 5]$ le minimum est à l'indice 2 (c'est 1) et le maximum à l'indice 1 (c'est 6) et comme le minimum est après le maximum ce n'est pas le saut maximal.

- Q21**– Écrire une fonction `sautmax_naif` qui renvoie un saut de valeur maximale dans un tableau de taille `n` en testant tous les couples (i,j) tels que $0 \leq i \leq j < n$.

```

1 double valeur(double tab[], int i, int j, int n)
2 {
3     assert(0 <= i && i <= j && j < n);
4     return tab[j] - tab[i];
5 }
```

- Q22**– Quelle est la complexité de la fonction `sautmax_naif` en fonction de la taille `n` du tableau ?

Il y a deux boucles `for` imbriquées et les deux sont exécutés au plus `n` fois, les opérations à l'intérieur de ces boucles sont toutes des opérations élémentaires, donc la complexité est en $\mathcal{O}(n^2)$

- Q23**– Au lieu de rechercher la valeur maximale d'un saut noté v_m , on peut vouloir récupérer deux indices (i_m, j_m) tels que le saut (i_m, j_m) soit de valeur v_m . Pour cela, on propose d'écrire une fonction `indices_sautmax` qui prendra en plus du tableau et de sa taille deux pointeurs vers des entiers `im` et `jm` qui seront modifiés par la fonction afin qu'ils contiennent après appel les indices d'un saut de valeur maximale. La signature de la fonction est alors :
`void indices_sautmax(double tab[], int n, int *im, int *jm)`, c'est à dire qu'elle ne renvoie rien mais modifiera le contenu des pointeurs `im` et `jm`. Ecrire cette fonction.

```

1 void indices_sautmax(double tab[], int n, int *im, int *jm)
2 {
3     double smax = 0;
4     double sij = 0;
5     for (int i = 0; i < n; i++)
6     {
7         for (int j = i; j < n; j++)
8         {
9             sij = valeur(tab, i, j, n);
10            if (sij > smax)
11            {
12                *im = i;
13                *jm = j;
14                smax = sij;
15            }
16        }
17    }
18 }

```

■ Partie II : Résolution avec une méthode diviser pour régner

On propose maintenant d'utiliser une méthode diviser pour régner afin de calculer la valeur maximale d'un saut. On note n la taille du tableau t et $p = \lfloor \frac{n}{2} \rfloor$ (où $\lfloor \cdot \rfloor$ désigne la partie entière). On souhaite calculer :

- (i_g, j_g) un saut de valeur maximale lorsque $j_g < p$ (c'est à dire un saut maximal dans la moitié gauche)
- (i_d, j_d) un saut de valeur maximale lorsque $i_d \geq p$ (c'est à dire un saut maximal dans la moitié droite)
- (i_m, j_m) un saut de valeur maximal lorsque $i_m < p < j_m$ (c'est à dire un saut maximal dont le premier indice est dans la moitié gauche et le second dans la moitié droite)

Q24— Justifier qu'un saut de valeur maximale du tableau t est nécessairement un des trois ci-dessus. On pourra noter (i, j) un saut maximal puis illustrer le raisonnement à l'aide d'un schéma en faisant figurer les divers cas possibles pour les positions relatives des entiers i , j et p .

Un saut de valeur maximal (i, j) du tableau t est tel que $i \leq j$ trois situations sont donc possibles en fonction de la position relative de i , j et p :

- $i \leq j < p$ et donc le saut maximal (i, j) se situe dans la moitié gauche du tableau.

 $0 \leq i \leq j < p$ et donc le saut maximal (i, j) se situe dans la moitié gauche du tableau.

- $p \leq i \leq j$ et donc le saut maximal (i, j) se situe dans la moitié droite du tableau.

 $p \leq i \leq j$ et donc le saut maximal (i, j) se situe dans la moitié droite du tableau.

- $0 \leq i < p < j \leq n$ c'est à dire que le saut « traverse » le milieu du tableau.

 Ce qui correspond bien aux trois situations décrites ci-dessus.

Q25— Justifier que i_m est nécessairement l'indice d'une valeur minimale dans la moitié gauche de t (on admettra que de même j_m est nécessairement l'indice d'une valeur maximale dans la moitié droite de t).

On raisonne par l'absurde, si tel n'était pas le cas, on aurait une valeur d'indice q dans la moitié gauche strictement inférieure à $t[i_m]$ et donc le saut (q, j_m) aurait une valeur supérieure au saut (i_m, j_m) ce qui contredit que (i_m, j_m) est un saut de valeur maximale.

Q26— Ecrire une fonction de signature `int min(double tab[], int a, int b)` qui prend en argument un tableau `tab`, ainsi que deux entiers `a` et `b` (avec `a ≤ b`) et renvoie l'indice d'un minimum de `tab` entre les deux indices `a` (inclus) et `b` (exclu).

On supposera dans la suite déjà écrite une fonction qui `max` qui prend les mêmes arguments et renvoie l'indice d'un maximum du sous tableau `{tab[a], ..., tab[b-1]}`


```

1  double min(double tab[], int a, int b)
2  {
3      assert(a < b);
4      double minv = tab[a];
5      for (int i = a + 1; i < b; i++)
6      {
7          if (tab[i] < minv)
8          {
9              minv = tab[i];
10         }
11     }
12     return minv;
13 }

```

Q27– Ecrire une fonction de signature `double sautmax_dpr(double tab[], int a, int b)` qui prend en argument un tableau `tab`, ainsi que deux entiers `a` et `b` (avec $a \leq b$) et renvoie la valeur d'un saut maximale dans `tab` entre les deux indices `a` (inclus) et `b` (exclu). Cette fonction doit être récursive et utiliser la méthode diviser pour régner. On pourra supposer déjà écrite une fonction `max3` qui renvoie le maximum des trois `double` donnés en argument.

```

1  double sautmax_dpr(double tab[], int a, int b)
2  {
3      // Renvoie le saut maximal du sous tableau t[a],...t[b-1]
4      // en utilisant une stratégie diviser pour régner
5      if (b - a < 2)
6      {
7          return 0;
8      }
9      int p = (b - a) / 2;
10     double smg = sautmax_dpr(tab, a, a + p);
11     double smd = sautmax_dpr(tab, a + p, b);
12     double ming = min(tab, a, a + p);
13     double maxd = max(tab, a + p, b);
14     double smax = max3(smg, smd, maxd - ming);
15     return smax;
16 }

```

Q28– Déterminer la complexité de la fonction `sautmax_dpr`.

On obtient l'équation de complexité $C(2n) = 2C(n) + \mathcal{O}(n)$ en effet on résout deux sous problèmes de taille deux fois plus petite et on doit ensuite calculer un minimum et un maximum d'une liste de taille n et ces opérations sont en $\mathcal{O}(n)$. On suppose sans perdre de généralité que $n = 2^k$ et on note $u_k = C(2^k)/2^k$, en divisant l'équation de complexité par 2^{k+1} on obtient alors :

$$u_{k+1} \leq u_k + \frac{M2^k}{2^{k+1}} \text{ donc, } u_{k+1} \leq u_k + M \times \frac{1}{2} \text{ et en sommant pour } i = 0 \text{ à } k \text{ on obtient :}$$

$$u_k \leq u_0 + M'k$$

$$C(n) \leq n (C(1) + M' \log n)$$

$$\text{Et donc } C(n) \in \mathcal{O}(n \log n).$$

■ Partie III : Résolution par programmation dynamique

On cherche maintenant à résoudre ce problème par programmation dynamique, et on adopte les notations suivantes :

- t est un tableau de taille n contenant des flottants
- $t[i]$ est l'élément d'indice i ($0 \leq i < n$) de t ,

- pour $0 < k \leq n$, t_k est le sous tableau $t[0], \dots, t[k-1]$
- m_k est l'indice d'un minimum de t_k pour $0 < k < n$.
- (i_k, j_k) est un saut de valeur maximale dans t_k pour $0 < k < n$.

Q29– Donner les valeurs de i_1 , j_1 et m_1

Comme le tableau ne contient qu'un seul élément (celui d'indice 0), $i_1 = 0, j_1 = 0$ et $m_1 = 0$.

Q30– Donner la relation de récurrence liant m_{k+1} , m_k et $t[k+1]$.

Si $t[k+1] < m_k$ alors $m_{k+1} = t[k+1]$ sinon $m_{k+1} = m_k$.

Q31– Justifier que la relation suivante est correcte :

$$(i_{k+1}, j_{k+1}) = \begin{cases} (i_k, j_k) & \text{si } t[k] - t[m_k] < t[j_k] - t[i_k] \\ (m_k, k) & \text{sinon} \end{cases}$$

Le saut de valeur maximal ayant k comme second indice est (m_k, k) et vaut $t[k] - t[m_k]$. En effet le second indice étant fixé on l'obtient en prenant le minimum du sous tableau t_k . On doit donc comparer ce nouveau saut avec le saut maximal du sous tableau t_k . Soit il est plus petit c'est à dire $t[k] - t[m_k] < t[j_k] - t[i_k]$ et donc $(i_{k+1}, j_{k+1}) = (i_k, j_k)$ ou bien il est plus grand et donc $(i_{k+1}, j_{k+1}) = (m_k, k)$.

Q32– Ecrire une fonction de signature `double sautmax_dyn(double tab[], int n)` qui prend en argument un tableau et sa taille et renvoie la valeur maximale d'un saut de ce tableau. On procédera *de façon ascendante* en utilisant les relations de récurrences de la question précédente et en calculant successivement les valeurs maximales de saut dans les sous tableaux t_1 puis $t_2, \dots$ jusqu'à t_n .

```

1  double sautmax_dyn(double tab[], int n)
2  {
3      int i, j, m;
4      i = 0;
5      j = 0;
6      m = 0;
7      for (int k = 1; k < n; k++)
8      {
9          // Mettre à jour la valeur maximale du saut
10         if (tab[k] - tab[m] >= valeur(tab, i, j, n))
11         {
12             i = m;
13             j = k;
14         }
15         // Mettre à jour l'indice du minimum
16         if (tab[k] < tab[m])
17         {
18             m = k;
19         }
20     }
21     return valeur(tab, i, j, n);
22 }
```

Q33– Déterminer la complexité de la fonction `sautmax_dyn`

La fonction parcourt le tableau à l'aide d'une boucle `for` qui ne contient que des opérations élémentaires, la complexité est donc linéaire en fonction de la taille du tableau.